RAMSES Manual: DCTL MATLAB

February 12, 2014

1 Introduction

This manual shows how to design and implement a discrete controller in Matlab and interface it with RAMSES. The controller is activated after $T_{shift}(s)$ and then it acts every $T_{action}(s)$.

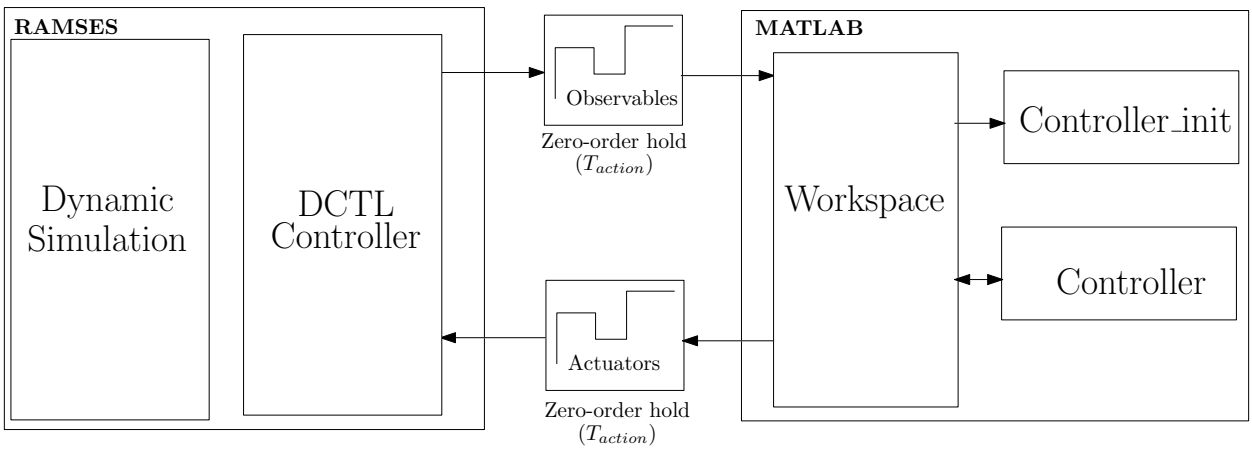


Figure 1: RAMSES to MATLAB connection

During the simulation RAMSES passes to the workspace of MATLAB the observable and actuator values needed. Then it calls the Controller function which reads the observable and actuator values from the workspace, calculates the new actuator values and puts them to the workspace. Finally, RAMSES receives the actuator values and applies the changes in the simulation.

The user needs to provide a description of the Controller to RAMSES as detailed in Section 2 which includes the observables, the actuators, the T_{shift} and T_{action} times.

Then, the user needs to provide two functions in MATLAB for the controller as detailed in Section 3. The first function is used for initialization at the beginning of the simulation and the second is the function called every T_{action}.

2 RAMSES Controller Description

In the data files of RAMSES the user needs to define the DCTL controller. The structure with all the available settings appears in Fig. 2.

```
DCTL MATLAB NameOfController T_shift(s) T_action(s)
OBSERVABLES
  V BusName (pu)
  INJ InjName P(MW)/Q(MVAr)
  SYN SynMachName P(MW)/Q(MVAr)/W(pu)
ACTUATORS
  INJ InjName ParName
  EXC SynMachName ParName
  TOR SynMachName ParName
  DCTL DCTLName ParName;
```

Figure 2: RAMSES DCTL data definition

In the example of Fig. 3, the name of the controller is *LoadSchedCtrl*, it will start acting after 20 s, it will act every 100 s. The observables are the voltage on bus 72 and the active and reactive power of injector L72 (a restorative load connected on bus 72). The actuators are the P0 and Q0 of the specific load.

```
DCTL MATLAB LoadSchedCtrl 20 100
OBSERVABLES
  V 72
  INJ L72 P
  INJ L72 Q
ACTUATORS
  INJ L72 P0
  INJ L72 Q0
  DCTL 1_1041 V0 ;
```

Figure 3: RAMSES DCTL example

3 MATLAB Controller Description

The OBSERVABLES and ACTUATORS selected in RAMSES DCTL data are transfered to MATLAB's workspace with the following naming convention:

RAMSES	MATLAB workspace variable
OBSERVABLES	
`V BusName`	`O_BusName_V`
`INJ InjName P/Q`	`O_InjName_P/Q`
`SYN SynMachName P/Q/W`	`O_SynMachName_P/Q/W`
ACTUATORS	
`INJ InjName ParName`	`U_InjName_ParName`
`EXC SynMachName ParName`	`U_SynMachName_ParName`
`TOR SynMachName ParName`	`U_SynMachName_ParName`
`DCTL DCTLName ParName`	`U_DCTLName_ParName`

For the example of Fig. 3 the variable names are:

RAMSES	MATLAB workspace variable
OBSERVABLES	
V 72	O_72_V
INJ L72 P	O_L72_P
INJ L72 Q	O_L72_Q
ACTUATORS	
INJ L72 P0	U_L72_P0
INJ L72 Q0	U_L72_Q0
DCTL 1_1041 V0	U_1_1041_V0

WARNING: Do not use the same name for different equipment because it could overwrite one another. For example, one injector with name XXXX modifying parameter with name YY will conflict with a DCTL with name XXXX modifying it's own parameter named YY. Symbols like *,+,-,/, etc. should be avoided in names because matlab does not allow them.

The user then needs to define two functions. One should have the same name as *NameOfController*. This function calls should read the OBSERVABLES and current ACTUATOR values from the workspace and overwrite the actuator variable with the new setpoints. The other function should be named *NameOfController_init* and is called only once at the initialization of the simulation. In this function the user might want to set-up his own variables, data structures etc.

Based on the example of Fig. 3, we can define the following functions:

```
function LoadSchedCtrl_init
```

and:

```
function LoadSchedCtrl

% First the OBSERVABLES and ACTUATORS are read into variables from the workspace
t=evalin('base','t'); % simulation time
O_72_V=evalin('base','O_72_V'); % Bus voltage
U_L72_P0=evalin('base','U_L72_P0');
U_L72_Q0=evalin('base','U_L72_Q0');

% Then the controller logic acts
if O_72_V < 1.0
   % The new actuator values are exported to the workspace
   assignin('base', 'U_L72_P0',U_L72_P0*0.99);
   assignin('base', 'U_L72_Q0',U_L72_Q0*0.99);
end
```

This controller has a very simple logic: As long as the voltage on bus 72 is lower than 1 pu, it sheds active and reactive load by adjusting the P0 and Q0 variables of the restorative load L72 attached on that bus.

4 Examples

The following examples are based on the Nordic32 system [1].

4.1 Example 1: Very simple load shedding logic

This is the controller described in the previous sections. So, in the data files of RAMSES we add a DCTL record:

```
DCTL MATLAB LoadSchedCtrl 20 100
OBSERVABLES
  V 72
  INJ L_72 P
  INJ L_72 Q
ACTUATORS
  INJ L_72 P0
  INJ L_72 Q0 ;
```

In the path of MATLAB we add two files. *LoadSchedCtrl_init.m*:

```
function LoadSchedCtrl_init
```

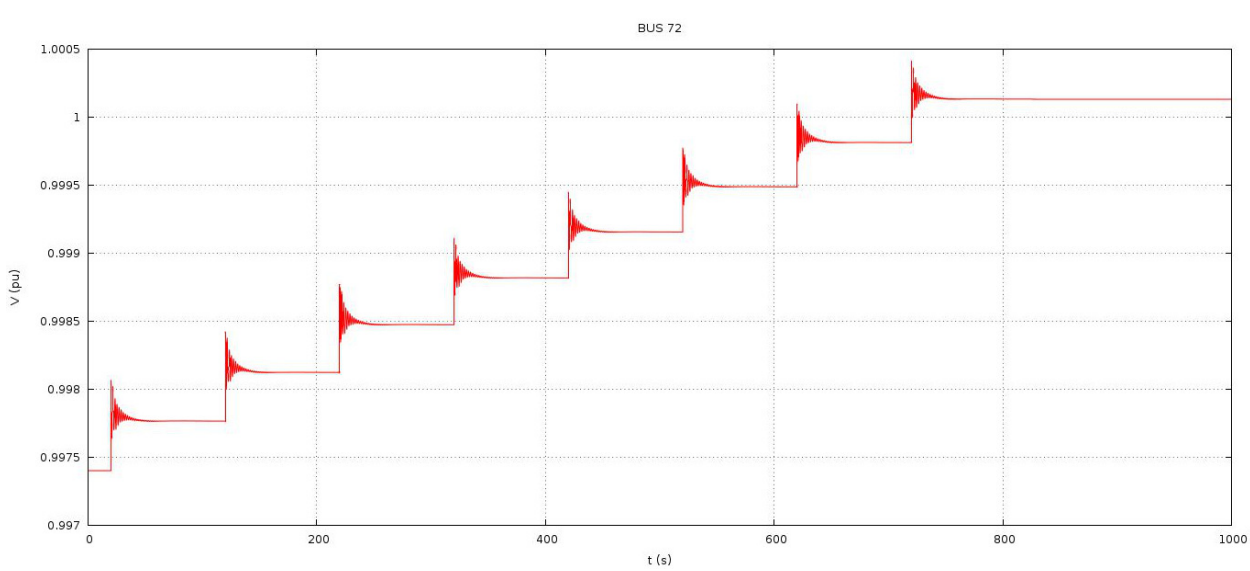
and *LoadSchedCtrl.m*:

```
function LoadSchedCtrl

% First the OBSERVABLES and ACTUATORS are read into variables from the workspace
t=evalin('base','t'); % simulation time
O_72_V=evalin('base','O_72_V'); % Bus voltage
U_L_72_P0=evalin('base','U_L_72_P0');
U_L_72_Q0=evalin('base','U_L_72_Q0');

% Then the controller logic acts
if O_72_V < 1.0
   % The new actuator values are exported to the workspace
   assignin('base', 'U_L_72_P0',U_L_72_P0*0.99);
   assignin('base', 'U_L_72_Q0',U_L_72_Q0*0.99);
end
```

The disturbance used for this is *nothing.dst*:

```
   0.000 CONTINUE SOLVER BD 0.020 0.001 0.0 ALL
1000.000 STOP
```

The plot of bus 72 voltage is:



4.2 Example 2: Synchronous machine exciter set-point

This is the controller described in the previous sections. So, in the data files of RAMSES we add a DCTL record:

```
DCTL MATLAB ExcVsetCtrl 20 100
OBSERVABLES
  V g20
ACTUATORS
  EXC g20 V0 ;
```

In the path of MATLAB we add two files. ***ExcVsetCtrl*** _ *init.m*:

```
function ExcVsetCtrl_init
```

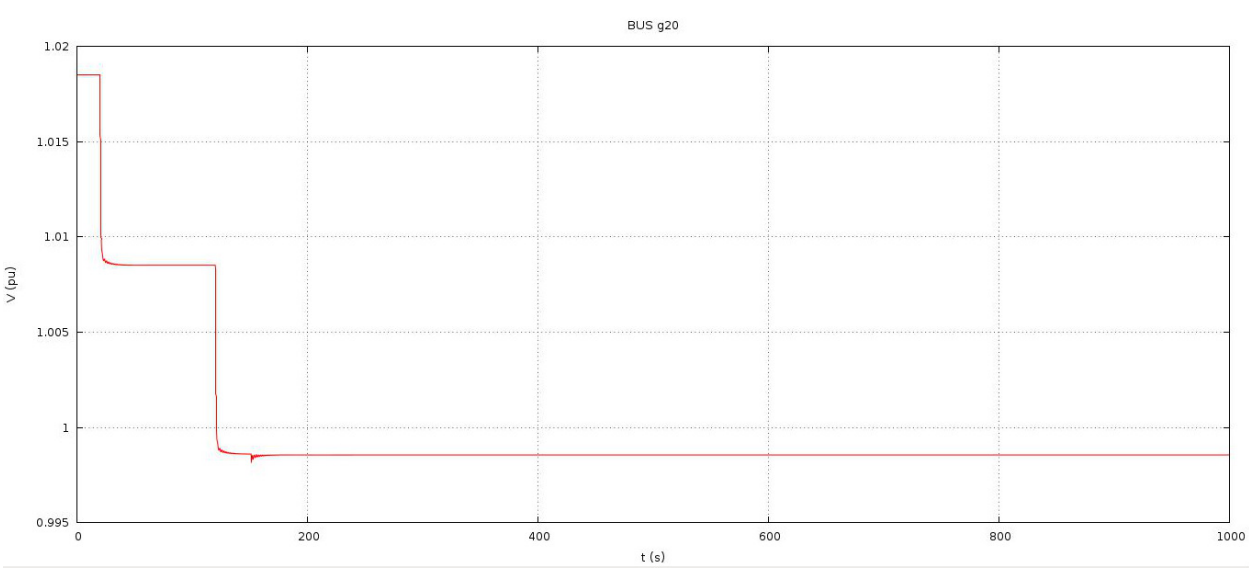
and ***ExcVsetCtrl*** *.m*:

```
function ExcVsetCtrl

% First the OBSERVABLES and ACTUATORS are read into variables from the workspace
t=evalin('base','t'); % simulation time
O_g20_V=evalin('base','O_g20_V'); % Bus voltage
U_g20_V0=evalin('base','U_g20_V0'); % Current exciter set-point

% Then the controller logic acts
if O_g20_V > 1.0
   % The new actuator values are exported to the workspace
   assignin('base', 'U_g20_V0',U_g20_V0*0.99); % New exciter set-point
end
```

The disturbance used for this is *nothing.dst*:

```
   0.000 CONTINUE SOLVER BD 0.020 0.001 0.0 ALL
1000.000 STOP
```

The plot of bus g20 voltage is:



5 Nordic32: Available control parameters

Please check [1] for the full model description.

5.1 LOAD

P0	Initial active power of exponential load
Q0	Initial reactive power of exponential load
V0	Initial terminal voltage of exponential load

5.2 EXC GENERIC1

V0	Initial terminal voltage of exciter

5.3 TOR HYDRO_GENERIC1

P0	Initial active power of synchronous machine

6 Hints

6.1 Matlab history

To keep a history for use in Matlab the user needs to define them in the init file and put them in the workspace. For example:

```
function ExcVsetCtrl_init
myHistory=zeros(3,3)
% export to workspace
assignin('base', 'myHistory',myHistory);
```

And to update them later on, in the controller function:

```
function ExcVsetCtrl

% read into variables from the workspace
myHistory=evalin('base','myHistory');
...
DO SOMETHING
UPDATE myHistory
...
% export back to workspace
assignin('base', 'myHistory',myHistory);
```

6.2 Workspace monitoring

Under Linux the user cannot access directly or monitor the workspace except from within the MATLAB files. Under windows though, the user can issue the command *workspace* in the MATLAB command window appearing, thus being able to overview the workspace variables.

6.3 Error message: Can't start MATLAB engine

This is an error due to MATLAB. It means it was **not** set up correctly as a COM service. Please revisit http://goo.gl/1sZXwz

References

[1] T. Van Cutsem, "Description, modeling and simulation results of a test system for voltage stability analysis," University of Liège, Internal report, 2013. [Online]. Available: http://hdl.handle.net/2268/141234